# Studio Nx

JS 110 — Transport.setTempo()

2026-10-14

# JS 110 — `Transport.setTempo()` (재생 중 템포 변경)

| 항목 | 내용 |
|---|---|
| 파일 | `studionx/frontend/static/js/audio/scheduler.js` (같은 클래스에 메서드 추가) |
| 작업 | S1-05 오디오 엔진 기초: 일곱 번째 함수 (FR-202 시작 BPM, FR-202a 곡 중간 템포 변경) |
| 작성 | Claude (AI), 2026-10-14, 사용자 검토 |
| 브랜치 | `feature/phase1/s1-05-audio-engine` |
| 다음 | 메트로놈, 루프 |

## 1. 하는 일

재생을 멈추지 않고 BPM과 곡 중간 템포 변경점(`tempo_changes`)을 바꿉니다. 재생 중이면 **지금 위치를 기억하고 다음 단계 경계로 올려 그곳부터 새 템포로 다시 예약**합니다.

| 호출 | 결과 |
|---|---|
| 재생 중 `setTempo(60, [])` (120→60) | 이후 단계 간격이 0.125초 → 0.25초 |
| 같은 자리에서 `setTempo(120, [{tick:3840,bpm:240}])` | 3840 뒤 간격이 0.125초 → 0.0625초 |
| 멈춰 있을 때 `setTempo` | 위치 그대로, 예약 안 함, 다음 재생에 반영 |
| 일시정지 중 `setTempo` | 위치 고정 유지 |

## 2. 코드

```
  setTempo(baseBpm, changes) {
    if (typeof baseBpm !== 'number' || !isFinite(baseBpm) || baseBpm <= 0) throw new RangeError('baseBpm
은 0 보다 큰 숫자여야 합니다: ' + baseBpm);
    const tick = this.positionTick();
    this.baseBpm = baseBpm;
    this.changes = changes != null ? changes : [];
    if (this.playing) {
      const from = this._alignUp(tick);
      this._clearTimer();
      this._anchor(from, this.clock.now() + this.startDelay);
      this.pump();
      this._scheduleTimer();
    }
    return this;
  }
```

**최신 문법으로 쓰면** (공부용)

```
  setTempo(baseBpm, changes = []) {
    if (!Number.isFinite(baseBpm) || baseBpm <= 0) throw new RangeError(`baseBpm 은 0 보다 큰 숫자여야 합
니다: ${baseBpm}`);
    // …나머지 같음
  }
```

| 최신 문법 | 실제 코드 | 뜻 |
|---|---|---|
| 기본 인자 `changes = []` | `changes != null ? changes : []` | 값이 없으면 빈 배열을 씁니다 |
| `Number.isFinite(x)` | `typeof x !== 'number' \|\| !isFinite(x)` | 숫자이면서 NaN·Infinity가 아닌지 한 번에 봅니다 |
| 템플릿 문자열 | `'…' + x` | 글자와 값을 잇습니다 |

## 3. 왜 이렇게

| 선택 | 이유 |
|---|---|
| 위치를 **바꾸기 전에** 읽음 | `positionTick()` 은 지금 `baseBpm`·`changes` 로 계산합니다. 먼저 위치를 구해 두고 템포를 바꿔야 옛 템포 기준 위치가 나옵니다 |
| 재생 중이면 단계 경계로 올림 | `resume` 과 같은 방식입니다. 커서가 이미 새 단계 경계(1/16박)에 맞춰져 있어, 이후 단계도 격자에 맞습니다. 대신 재생선이 최대 한 단계 앞으로 살짝 뛸 수 있습니다 |
| 커서를 다시 잡고 `pump()` | 새 템포로 다시 계산해야 하므로, 옛 커서의 미래 시각은 버리고 지금 위치부터 새로 잡습니다 |
| 이미 예약된 창은 옛 템포로 울림 | 커서 이후(최대 lookahead 0.1초)는 이미 예약되어 취소할 수 없습니다. 그 뒤부터 새 템포가 적용됩니다 (문서에 남김) |
| 멈춰 있을 때는 다시 잡지 않음 | `positionTick` 이 `originTick` 을 돌려주므로 위치가 그대로입니다. 다음 `start`·`resume` 이 새 템포를 씁니다 |
| `baseBpm` 을 먼저 검사 | 멈춰 있을 때는 `pump()` 를 안 불러 105의 검사가 안 돌 수 있어 여기서 직접 검사합니다 |

## 4. 시험 (Node에서 직접 실행, 18개 통과 · 가짜 시계·가짜 타이머)

| 확인 | 결과 |
|---|---|
| 재생 중 120→60: 이후 tick이 240씩, 시간 간격이 0.25초 | ✓ |
| 변경 전 간격이 0.125초였는지 | ✓ |
| 변경 위치가 단계 경계에 맞고 크게 안 뛰는지 | ✓ |
| 멈춰 있을 때: 예약 안 함, 위치 유지, 필드만 갱신 | ✓ |
| 일시정지 중: 위치 고정 유지, `playing` 그대로 | ✓ |
| 재생 중 템포 변경점 추가(3840 뒤 간격 0.0625초) | ✓ |
| `baseBpm` 0·NaN 이면 `RangeError` | ✓ |